OOP Lab

Mobile Repairing Shop Management System

Mid Project Report

Instructor

Ma'am Rimsha Afzal

Submitted by:

Tooba Umair

DEPARTMENT OF COMPUTER SCIENCE

University of Engineering and Technology, Lahore

Contents

1. Introduction 3
2. Features 3
2.1 Core Features 3
2.2 Technical Features 3
3. ORM – Object Relational Mapping 3
3.1 How ORM Works in This Project 4
3.2 ORM Table Mapping 4
3.3 Example – Loading Customer Data 4
4. Static Functions 4
4.1 Static ID Generators 5
4.2 Example Code 5
4.3 Static DL Lists 5
5. Encapsulation 5
5.1 Pattern Used 5
5.2 Encapsulated Fields Per Class 6
6. Validation 6
6.1 Validation Rules Table 6
6.2 ID Existence Validation 7
7. Association 7
7.1 Associations in This System 7
7.2 Association Example in Code 8
8. Aggregation and Composition 8
8.1 Aggregation (Weak Relationship) 8
8.2 Composition (Strong Relationship) 9
8.3 Summary Diagram 9
9. Data Layers (Three-Layer Architecture) 9
9.1 Layer Overview 9
9.2 How the Layers Communicate 9
9.3 DatabaseHelper Class (DL Layer) 10
9.4 Data Flow Example – Adding a Customer 10
10. Test Cases 10
10.1 Customer Test Cases 10
10.2 Employee Test Cases 11
10.3 Order Test Cases 11
10.4 Service and SparePart Test Cases 12
11. Conclusion 12

1. Introduction

The Mobile Repairing Shop Management System is a console-based C# application built to manage the day-to-day operations of a mobile phone repair shop. It follows a clean three-layer architecture and applies core Object-Oriented Programming (OOP) principles throughout.

The system handles five main entities:

- Employee – Technicians who work at the shop
- Customer – Clients who bring in phones for repair
- Order – A repair job placed by a customer
- Service – A specific repair action (e.g., screen replacement)
- SparePart – Physical parts used during a service

Each entity supports full CRUD operations (Create, Read, Update, Delete) backed by a MySQL database. The project connects to the database via a centralized DatabaseHelper class in the Data Layer (DL), keeping data access logic completely separate from business logic.

2. Features

2.1 Core Features

- Full CRUD for Employees, Customers, Orders, Services, and Spare Parts
- Filter orders by status: pending, in-progress, delivered, completed
- View all orders placed by a specific customer
- View all customers handled by a specific employee
- View all spare parts used in a specific service
- View all services associated with an order

2.2 Technical Features

- Three-layer architecture: UI, BL (Business Logic), DL (Data Layer)
- MySQL database integration with a centralized DatabaseHelper
- Auto-generated IDs for every entity (e.g., CU001, EM001, OR001)
- Input validation on every setter method returning a boolean result
- In-memory lists in each DL class for fast read operations
- Data loaded fresh from the database at the start of every menu loop

3. ORM – Object Relational Mapping

The project uses a manual ORM pattern. Each Data Layer class maps database table rows directly to C# objects using MySqlDataReader. There is no third-party ORM framework; all SQL queries are hand-written and executed through the DatabaseHelper class.

3.1 How ORM Works in This Project

Every DL class (CustomerDL, EmployeeDL, OrderDL, ServiceDL, SparePartDL) has a loadData() method that:

1. Opens a MySqlConnection via DatabaseHelper.getConnection()
2. Executes a SELECT query
3. Reads each row with MySqlDataReader
4. Creates a BL object from the row data
5. Adds the object to an in-memory List<T>

3.2 ORM Table Mapping

Database Table	C# Class (BL)	DL Class	Key Fields
customer	Customer	CustomerDL	CustomerID, CustomerName, CNIC, PhoneModel, EmployeeID
employee	Employee	EmployeeDL	EmployeeID, EmployeeName, EmpContact, Salary, HiredDate
orders	Order	OrderDL	OrderID, OrderType, Condition, Status, Cost, CustomerID
service	Service	ServiceDL	ServiceID, ServiceName, Cost, TimeRequired
sparepart	SparePart	SparePartDL	PartID, PartName, Price, Stock
orderservice	Order <-> Service	OrderDL / ServiceDL	OrderID, ServiceID (join table)
servicesparepart	Service <-> SparePart	ServiceDL / SparePartDL	ServiceID, PartID (join table)

3.3 Example – Loading Customer Data

```
string query = "SELECT CustomerID, CustomerName, CustomerCNIC, NoOfOrder, PhoneModel,
EmployeeID FROM customer";
var command = new MySqlCommand(query, connection);
using (var reader = command.ExecuteReader()) {
    while (reader.Read()) {
        Customer C = new Customer(reader.GetString(0), reader.GetString(1), ...);
        customers.Add(C);
    }
}
```

4. Static Functions

Static methods in this project serve two main purposes: generating unique IDs and acting as utility/helper functions that do not need an object instance.

4.1 Static ID Generators

Each BL class has a private static counter and a static method to generate a unique ID every time a new object is created:

Class	Static Counter	Generated ID Format	Example Output
Customer	customer_id_no	"CU" + 3-digit number	CU001, CU002, CU003
Employee	employee_id_no	"EM" + 3-digit number	EM001, EM002
Order	order_id_no	"OR" + 3-digit number	OR001, OR002
Service	service_id_no	"SE" + 3-digit number	SE001, SE002
SparePart	sparepart_id_no	"SP" + 3-digit number	SP001, SP002

4.2 Example Code

```
private static int customer_id_no = 0;


public static string GenerateCustomerID() {
    customer_id_no++;
    return "CU" + customer_id_no.ToString("D3");
}
```

Why static? Because ID generation does not depend on any specific customer object. It tracks a shared counter across all instances, which is exactly what static is designed for.

4.3 Static DL Lists

Each DL class also holds a static List<T> for in-memory storage:

```
public static List<Customer> customers = new List<Customer>();
```

This means any part of the program can access the full list without creating a DL object. It acts like a lightweight in-memory cache that stays in sync with the database.

5. Encapsulation

All BL classes use private fields with public getter and setter methods. This is a textbook example of encapsulation – the data is hidden and can only be accessed or changed through controlled methods.

5.1 Pattern Used

```
private string customerName;


public string getCustomerName() {
```

```
    return customerName;
}

public bool setCustomerName(string customerName) {
    if (string.IsNullOrWhiteSpace(customerName)) return false;
    this.customerName = customerName.Trim();
    return true;
}
```

Notice that the setter returns a bool. This combines encapsulation with validation in a single clean method. If the value is invalid, the field is NOT changed, protecting the object's integrity.

5.2 Encapsulated Fields Per Class

Class	Private Fields
Customer	cusId, employeeId, customerName, customerCNIC, noOfOrder, phoneModel, placedOrders
Employee	employeeID, employeeName, empContact, hiredDate, salary, noOfCustomerHandled, handledCustomer
Order	orderID, orderType, orderCondition, customerID, status, cost, usedService
Service	serviceId, serviceName, cost, timeRequired, usedParts, usedInOrder
SparePart	partID, partName, price, stock, usedInServices

6. Validation

Every setter method validates its input before assigning it. If the input is invalid, the method returns false and leaves the field unchanged. This prevents bad data from entering the system.

6.1 Validation Rules Table

Class	Field	Validation Rule
Customer	customerName	Must not be null or whitespace
Customer	customerCNIC	Must not be null, whitespace, or shorter than 13 characters
Customer	noOfOrder	Must be >= 0
Customer	phoneModel	Must not be null or whitespace
Customer	employeeId	Must exist in EmployeeDL list (FK check)
Employee	employeeName	Must not be null or whitespace

Class	Field	Validation Rule
Employee	empContact	Must not be null, whitespace, or shorter than 10 characters
Employee	hiredDate	Must not be a future date
Employee	salary	Must be >= 0
Order	orderType	Must not be null or whitespace
Order	orderCondition	Must not be null or whitespace
Order	cost	Must be >= 0
Order	status	Must not be null or whitespace
Service	serviceName	Must not be null or whitespace
Service	cost	Must be >= 0
Service	timeRequired	Must not be null or whitespace
SparePart	partName	Must not be null or whitespace
SparePart	price	Must be >= 0
SparePart	stock	Must be >= 0

6.2 ID Existence Validation

When updating a record, the setter for the ID field checks that the ID already exists in the DL list before accepting it. This prevents updating non-existent records:

```
public bool setEmployeeID(string employeeID) {
    if (!EmployeeDL.isIdExist(employeeID)) return false;
    this.employeeID = employeeID.Trim();
    return true;
}
```

7. Association

Association represents a relationship between two classes where one class uses or references another, but both can exist independently. In this project, association is implemented using foreign key IDs and by storing lists of related objects inside a class.

7.1 Associations in This System

From Class	To Class	Type	How It Is Stored	Multiplicity
Employee	Customer	Association	Employee holds List<Customer> handledCustomer	1 Employee : Many Customers

From Class	To Class	Type	How It Is Stored	Multiplicity
Customer	Order	Association	Customer holds List<Order> placedOrders	1 Customer : Many Orders
Order	Service	Association	Order holds List<Service> usedService	Many Orders : Many Services
Service	SparePart	Association	Service holds List<SparePart> usedParts	Many Services : Many Parts
SparePart	Service	Back-ref Association	SparePart holds List<Service> usedInServices	Many Parts : Many Services
Customer	Employee	Referencing	Customer stores employeeId (FK) as a string	Many Customers : 1 Employee

7.2 Association Example in Code

```
// Inside Employee class
private List<Customer> handledCustomer = new List<Customer>();

// Inside Customer class
private string employeeId;  // stores FK reference
private List<Order> placedOrders = new List<Order>();
```

When the Employee object is loaded, the DL layer fetches all customers linked to that employee and populates the handledCustomer list.

8. Aggregation and Composition

Aggregation and Composition are two types of association that differ in how tightly objects are connected and whether child objects can live independently.

8.1 Aggregation (Weak Relationship)

In aggregation, the child object can exist independently of the parent. If the parent is deleted, the child still exists.

Relationship	Explanation
Service and SparePart	A SparePart (e.g., OLED Screen, Battery) exists in stock independently. If a Service is removed, the SparePart is not deleted.
Order and Service	A Service (e.g., Battery Swap) can exist in the system without being linked to any order. Deleting an order does not delete the service.
Employee and Customer	A Customer record exists independently. Deleting an employee does not delete the customer.

8.2 Composition (Strong Relationship)

In composition, the child cannot exist without the parent. The child's life cycle depends on the parent.

Relationship	Explanation
Customer and Order	When a new Customer is added, their Orders are created as part of the same transaction (via addCustomer flow in CustomerUI). The Order is meaningless without the Customer. This is shown in the code where Orders are created inside the addCustomer() method.

8.3 Summary Diagram

```
Employee  ----<>---- Customer  ----<*>---- Order  ----<>---- Service  ----<>----
SparePart
   (Aggregation)       (Composition)      (Aggregation)       (Aggregation)
```

Legend: <> = Aggregation | <*> = Composition

9. Data Layers (Three-Layer Architecture)

The project is cleanly divided into three layers. Each layer has a specific responsibility and does not mix concerns with other layers.

9.1 Layer Overview

Layer	Folder	Classes	Responsibility
UI Layer	/UI	CustomerUI, EmployeeUI, OrderUI, ServiceUI, SparePartUI, MenuUI	Takes input from user, displays output. Does not touch the database.
BL Layer	/BL	Customer, Employee, Order, Service, SparePart	Defines data fields, validation, and business rules. No database code.
DL Layer	/DL	CustomerDL, EmployeeDL, OrderDL, ServiceDL, SparePartDL, DatabaseHelper	Handles all database queries (SELECT, INSERT, UPDATE, DELETE). Maintains in-memory lists.

9.2 How the Layers Communicate

```
User types input
    -> UI Layer collects and validates display logic
        -> BL Layer creates/validates the object
            -> DL Layer saves or retrieves from MySQL database
```

9.3 DatabaseHelper Class (DL Layer)

DatabaseHelper is a centralized utility class in the DL layer that:

- Reads the connection string from App.config
- Opens a MySqlConnection
- Provides getData() for SELECT queries (returns DataTable)
- Provides Update() for INSERT, UPDATE, DELETE queries (returns rows affected)

```
public static int Update(string query) {
    using (var connection = getConnection())
    using (var command = new MySqlCommand(query, connection)) {
        return command.ExecuteNonQuery();
    }
}
```

9.4 Data Flow Example – Adding a Customer

Step	Layer	Action
1	UI	CustomerUI.addCustomer() reads name, CNIC, phone model, number of orders, and employee ID from console
2	BL	new Customer() object is created; setters validate each field and return false if invalid
3	UI	If validation passes, customer object is returned to Program.cs
4	DL	CustomerDL.addCustomerToList(C) runs an INSERT SQL query via DatabaseHelper.Update()
5	DL	The new Customer object is also added to the in-memory customers list

10. Test Cases

The following test cases cover the key validation rules and CRUD operations in the system.

10.1 Customer Test Cases

Test ID	Test Case	Input	Expected Result	Pass/Fail
TC-CU-01	Add customer with valid data	Name: Ali, CNIC: 4210112345671, Phone: Samsung S23, Orders: 1	Customer added successfully, ID generated (e.g., CU001)	Pass

Test ID	Test Case	Input	Expected Result	Pass/Fail
TC-CU-02	Add customer with empty name	Name: (blank)	setCustomerName() returns false, customer not added	Pass
TC-CU-03	Add customer with CNIC less than 13 chars	CNIC: 12345	setCustomerCNIC() returns false	Pass
TC-CU-04	Add customer with invalid Employee ID	EmployeeID: EM999 (not in system)	EmployeeDL.isIdExist() returns false, customer not added	Pass
TC-CU-05	Update customer with existing ID	ID: CU001, new name: Bilal	Record updated in DB and in-memory list	Pass
TC-CU-06	Delete customer with valid ID	ID: CU001	Record removed from DB and list	Pass
TC-CU-07	View orders of a customer	Customer ID: CU001	All linked Order objects displayed	Pass

10.2 Employee Test Cases

Test ID	Test Case	Input	Expected Result	Pass/Fail
TC-EM-01	Add employee with valid data	Name: Ahmed, Contact: 03001234567, Salary: 45000	Employee added, ID generated (EM001)	Pass
TC-EM-02	Add employee with short contact	Contact: 0300 (4 digits)	setEmpContact() returns false	Pass
TC-EM-03	Set future hired date	HiredDate: 2027-01-01	setHiredDate() returns false	Pass
TC-EM-04	Set negative salary	Salary: -5000	setSalary() returns false	Pass
TC-EM-05	View customers handled by employee	Employee ID: EM001	Lists all customers with that EmployeeID	Pass

10.3 Order Test Cases

Test ID	Test Case	Input	Expected Result	Pass/Fail
TC-OR-01	Add order with valid data	Type: Repair, Condition: Cracked, Cost: 5000	Order added, default status = pending	Pass
TC-OR-02	Add order with negative cost	Cost: -100	setCost() returns false	Pass

Test ID	Test Case	Input	Expected Result	Pass/Fail
TC-OR-03	Filter orders by status	Status: pending	Only pending orders displayed	Pass
TC-OR-04	View services of an order	Order ID: OR001	All linked Service objects displayed	Pass

10.4 Service and SparePart Test Cases

Test ID	Test Case	Input	Expected Result	Pass/Fail
TC-SE-01	Add service with valid data	Name: Screen Replacement, Cost: 5000, Time: 2 Hours	Service added, ID generated (SE001)	Pass
TC-SE-02	Add service with empty name	Name: (blank)	setServiceName() returns false	Pass
TC-SE-03	View spare parts of a service	Service ID: SE001	All linked SparePart objects displayed	Pass
TC-SP-01	Add spare part with valid data	Name: OLED Screen, Price: 4500, Stock: 15	Part added, ID generated (SP001)	Pass
TC-SP-02	Add spare part with negative price	Price: -200	setPrice() returns false	Pass
TC-SP-03	View services that use a spare part	Part ID: SP001	All linked Service objects displayed	Pass

11. Conclusion

The Mobile Repairing Shop Management System successfully demonstrates core OOP principles in a practical, real-world scenario. Here is a quick summary of what was applied:

OOP Concept	Where Applied
Encapsulation	All BL class fields are private, accessed only via get/set methods
Validation	Every setter validates input and returns false on failure
Static Methods	ID generators and DL in-memory lists are static
ORM	Manual mapping between MySQL rows and C# BL objects in each DL class
Association	Classes reference each other via foreign key IDs and List<T> collections

OOP Concept	Where Applied
Aggregation	Service-SparePart, Order-Service, Employee-Customer relationships
Composition	Customer owns Orders; Orders are created as part of the Customer workflow
Three-Layer Architecture	UI, BL, and DL layers are cleanly separated with clear responsibilities

The project is a solid foundation that can be extended with features such as billing, reports, user authentication, and a graphical user interface.